



دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده برق و کامپیوتر



یادگیری ماشین

تمرین شماره 5

نام و نام خانوادگی
سید محمد جزایری

شماره دانشجویی
810101399

بهمن 1404

فهرست

3.....	فهرست شکل‌ها
3.....	فهرست جدول‌ها
4.....	چکیده
5.....	پرسش 1 - عنوان پرسش
5.....	پرسش 2 - عنوان پرسش
5.....	پرسش 3 - عنوان پرسش
6.....	پرسش 4 - عنوان پرسش
11.....	پرسش 5 - عنوان پرسش
11.....	1-5. عنوان بخش
11.....	2-5. عنوان بخش
11.....	3-5. عنوان بخش
12.....	4-5. عنوان بخش
12.....	5-5. عنوان بخش
12.....	پرسش 6 - عنوان پرسش
14.....	پرسش 7 - عنوان پرسش
14.....	1-7. عنوان بخش
14.....	2-7. عنوان بخش
14.....	3-7. عنوان بخش
15.....	پرسش 8 - عنوان پرسش
16.....	پرسش 9 - عنوان پرسش
17.....	پرسش 10 - عنوان پرسش
23.....	مراجع

فهرست شکل‌ها

Fig 1. Polynomial regression on data_A.....	7
Fig 2. Polynomial degree VS. MSE on data_B.....	8
Fig 3. Ridge lambda VS. MSE on data_B.....	9
Fig 4. Training loss VS. epoch.....	18
Fig 5. Validation accuracy VS. epoch.....	19

فهرست جدول‌ها

NO TABLE

In this assignment we implement most of what we have learned about regression and neural networks to see their limitations and advantages.

پرسش 1 - عنوان پرسش

I will write the log-likelihood and set the derivative to zero.

$$L(\theta) = \prod_{i=1}^n \theta x_i^{-(\theta+1)}$$

$$\ell(\theta) = \ln L(\theta) = n \ln \theta - (\theta + 1) \sum_{i=1}^n \ln x_i$$

$$\frac{d\ell}{d\theta} = \frac{n}{\theta} - \sum_{i=1}^n \ln x_i = 0 \Rightarrow \hat{\theta} = \frac{n}{\sum_{i=1}^n \ln x_i}$$

Constraint note: since $\theta > 1$, if $\frac{n}{\sum_{i=1}^n \ln x_i} \leq 1$, the constrained maximizer is at the boundary

$$\theta \rightarrow 1^+$$

پرسش 2 - عنوان پرسش

I will take the derivative of $J(\alpha)$ with respect to α and then plug it in the SGD algorithm.

$$\frac{dJ}{d\alpha} = \frac{s-y}{s(1-s)} \cdot (-s(1-s)) = y - s$$

SGD with step size $\epsilon > 0$:

$$\alpha \leftarrow \alpha - \epsilon \frac{dJ}{d\alpha} = \alpha - \epsilon(y - s) = \alpha + \epsilon(s - y)$$

SGD update:

$$\alpha \leftarrow \alpha + \epsilon(s(x - \alpha) - y)$$

پرسش 3 - عنوان پرسش

Newton update:

$$w_1 = w_0 - H^{-1} \nabla J(w_0)$$

I will calculate the Hessian and the gradient and plug them in.

$$\nabla J(w) = 2X^T(Xw - y) + 2\lambda w$$

$$H = \nabla^2 J(w) = 2(X^T X + \lambda I)$$

Plug in:

$$w_1 = w_0 - [2(X^T X + \lambda I)]^{-1} [2X^T (Xw_0 - y) + 2\lambda w_0]$$

After simplifications:

$$w_1 = (X^T X + \lambda I)^{-1} X^T y$$

This value is the objective of the question.

پرسش 4 - عنوان پرسش

First I will illustrate the outputs and then analyze the

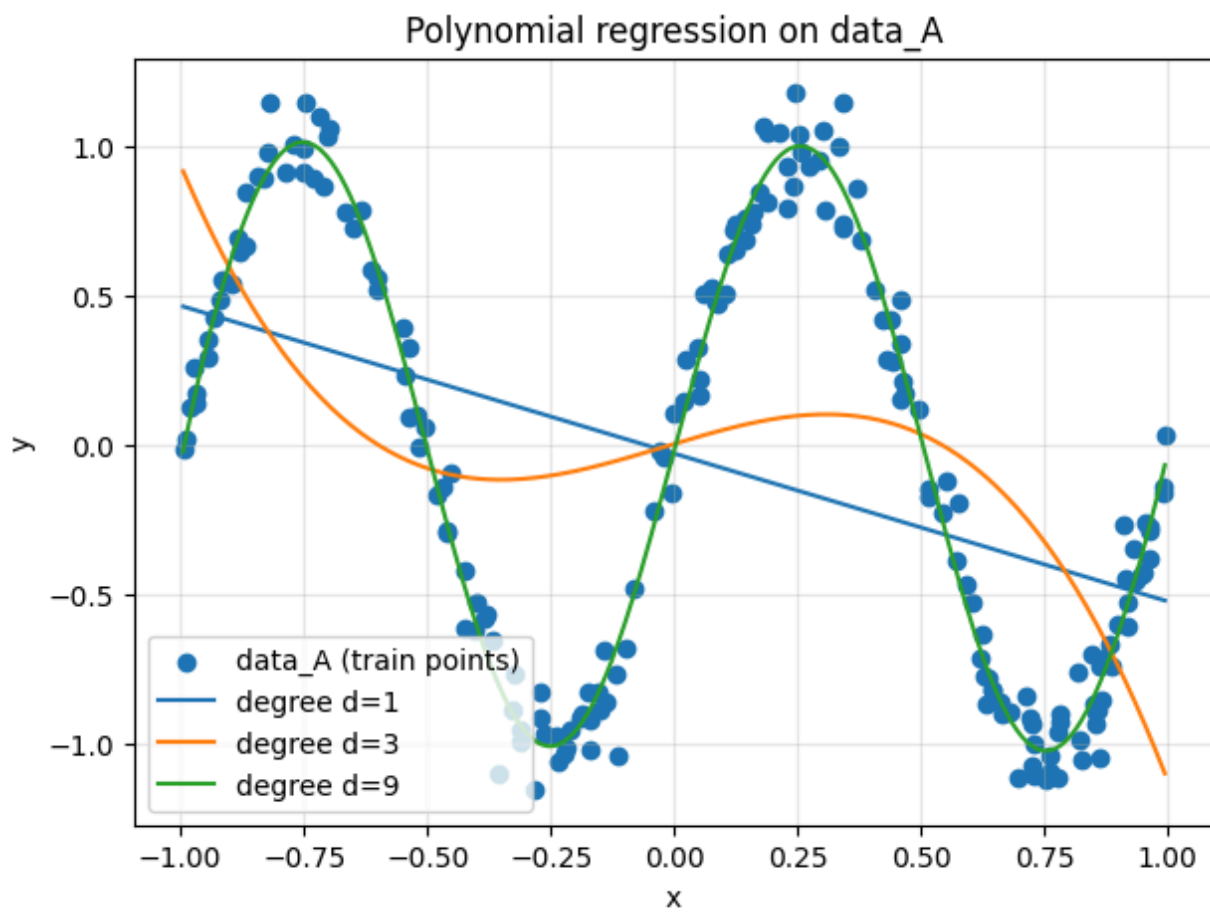


Fig 1. Polynomial regression on data_A

Best degree by validation MSE: 9
Train MSE at best degree: 0.03905685177655562
Val MSE at best degree: 0.07712854258160705

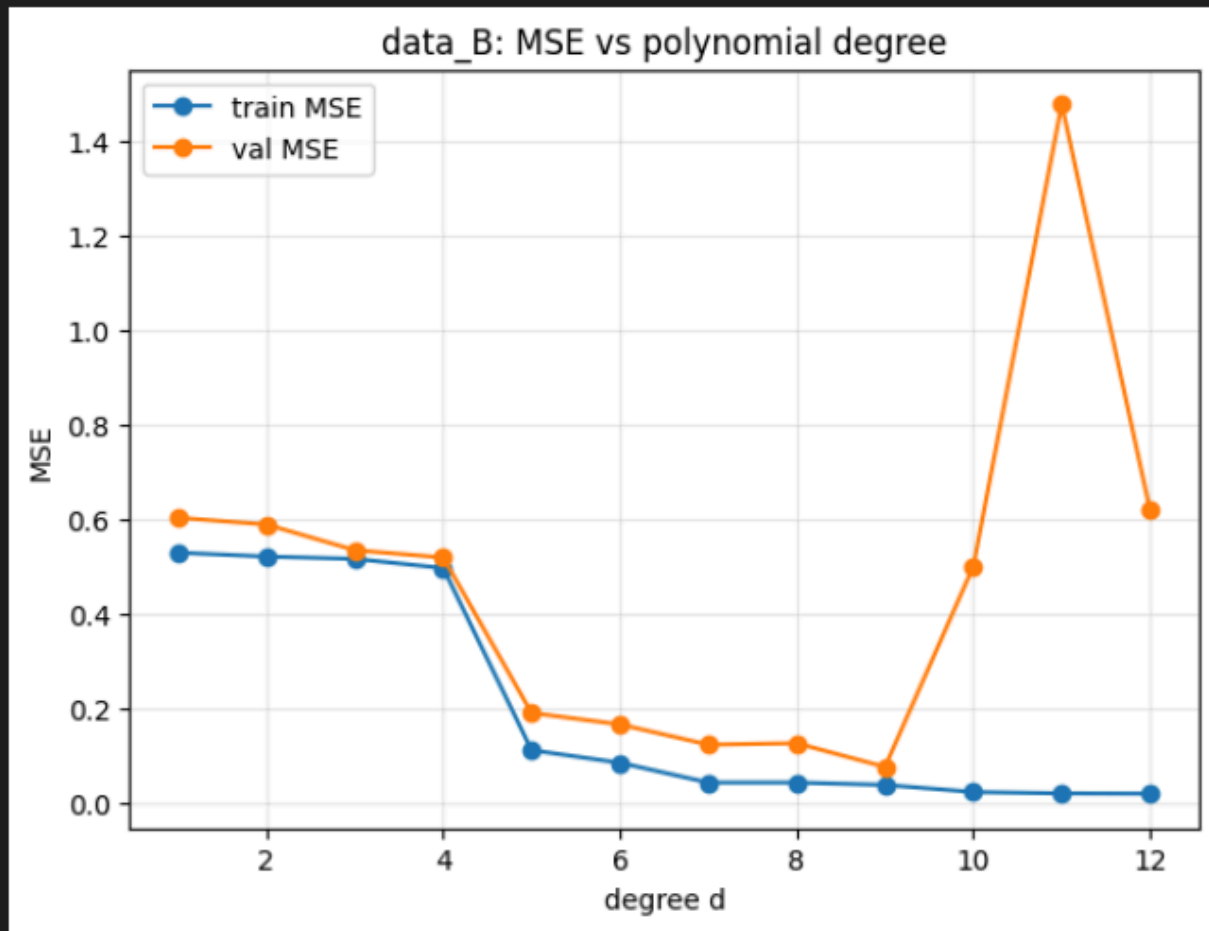


Fig 2. Polynomial degree VS. MSE on data_B


```

Best lambda by validation MSE: 0.0016608827826277166
Train MSE at best lambda: 0.08034471774439385
Val MSE at best lambda: 0.1445461933829057

```

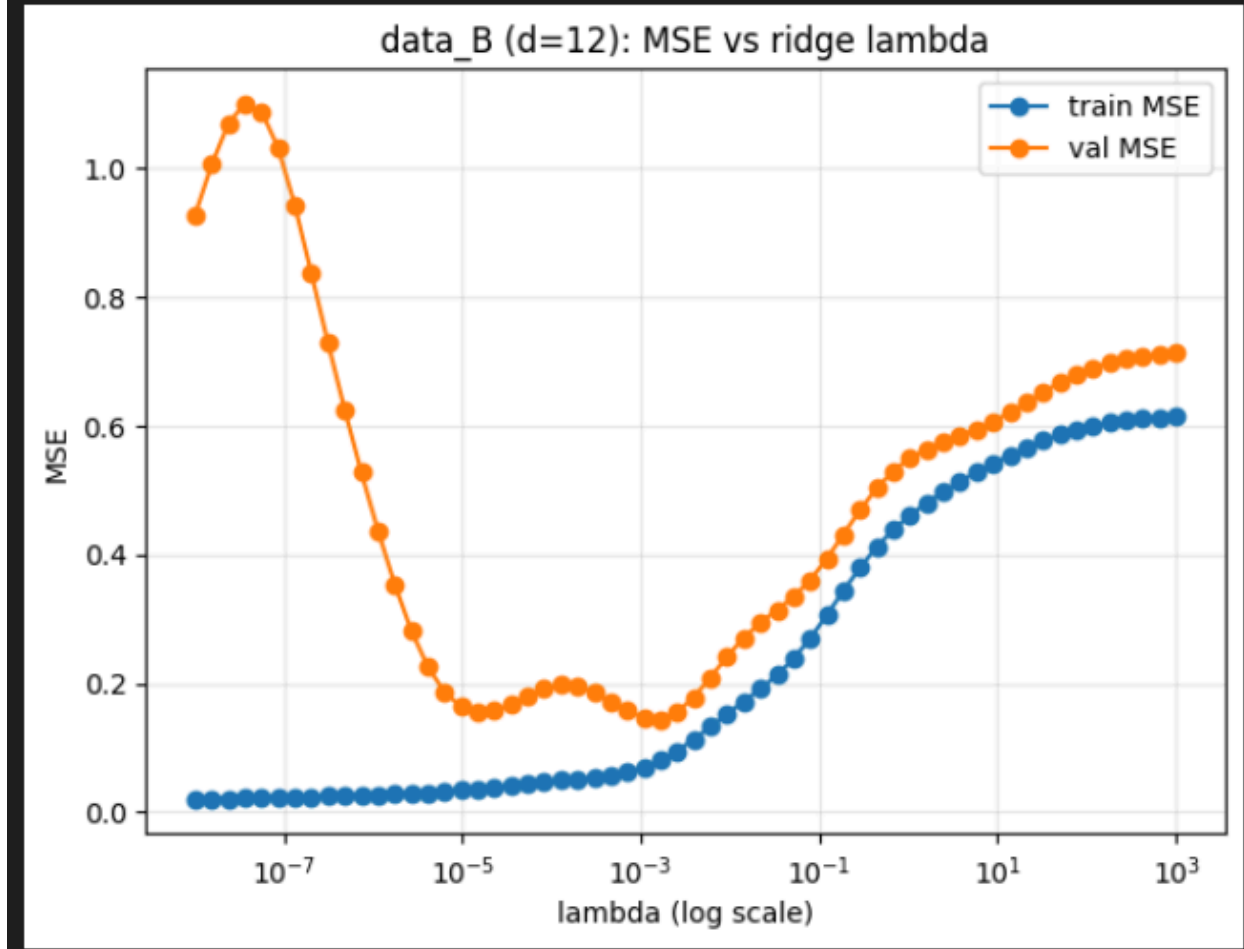


Fig 3. Ridge lambda VS. MSE on data_B

data_A

- **Degree 1 clearly underfits.** The true pattern is strongly curved/oscillatory, but $d=1$ is a straight line and misses the two main peaks and troughs almost everywhere.
- **Degree 3 also underfits.** It adds curvature, but still cannot capture the repeated up-down structure (it smooths through the oscillations and is systematically off near both peaks and troughs).
- **Degree 9 fits the visible pattern well:** it closely tracks the two-wave shape and stays near most points across the whole interval. I don't see strong wiggly behavior that would suggest obvious unnecessary complexity; at most, there's a *slight* edge/extrapolation sensitivity near the boundaries (common for higher-degree polynomials), but overall it matches the data's structure.

data_B: MSE vs degree

From our plot:

- **Underfitting range (clear): $d=1$ to $d=4$.** Both train and validation MSE are high (around ~ 0.5 – 0.6) and improve only slowly—model capacity is too small to represent the trend well.
- **Good trade-off range: roughly $d=7$ to $d=9$.**
 - Training MSE is low and continues to drop, but validation MSE is also low and near its minimum.
 - **Best validation is at $d=9$:**
 - Train MSE ≈ 0.0391
 - Val MSE ≈ 0.0771
- **Overfitting range: $d=10$ to $d=12$.** Training error keeps decreasing, but validation error jumps sharply (especially at $d=11$), which is classic high-variance behavior on a small training set.

data_B: Ridge at fixed $d=12$, MSE vs λ

What our curve shows as λ increases:

- **Training MSE increases steadily** with λ : stronger regularization shrinks coefficients more, so the model can't fit the training data as tightly.
- **Validation MSE is U-shaped:**
 - **Very small $\lambda \rightarrow$ high validation MSE** (strong overfitting / unstable high-degree fit).
 - **Moderate $\lambda \rightarrow$ lowest validation MSE** (best bias–variance trade-off).
 - **Very large $\lambda \rightarrow$ validation MSE rises again** (underfitting because the model is overly constrained).

From our reported best value:

- **Best $\lambda \approx 0.00166$**
 - Train MSE $\approx 0.0803 \approx 0.0803$
 - Val MSE $\approx 0.1445 \approx 0.1445$

Range where val MSE is minimized: around 10^{-3} (roughly a few $\times 10^{-4}$ to a few $\times 10^{-3}$ based on the bottom of the curve).

Comparison / takeaway across both datasets

- **Polynomial degree controls model complexity.** Increasing d lowers training error but can dramatically increase validation error once the model starts fitting noise (data_B shows this strongly at $d \geq 10$).
- **Why degree 9 behaves differently across datasets:** data_A has many points (200) and a smooth, repeatable pattern, so $d=9$ generalizes well. data_B has a tiny training set (20 points), so high degrees quickly become high-variance and overfit.

- **Ridge regularization helps high-degree models** by shrinking coefficients, which reduces the wiggles and sensitivity of a high-degree polynomial. It improves generalization for intermediate λ , but too much regularization increases bias and underfits.

پرسش 5 - عنوان پرسش

The perceptron update rule if the classification is wrong:

$$w \leftarrow w + yx$$

Classification rule:

$$\text{sign}(w^T x)$$

1-5. عنوان بخش

Prediction sign is $\text{sign}(w^T x) = \text{sign}([0, 5] \cdot [0, 1]) = \text{sign}(5) = +$, which is wrong.

Update:

$$w_{\text{new}} = w + yx = [0, 5] + (-1)[0, 1] = [0, 4]$$

2-5. عنوان بخش

With no bias term, any classifier is $\text{sign}(w^T x)$, and points (1,0) and (2,0) are on the same ray (same sign for any w) but have opposite labels here (negative vs positive). So it is not linearly separable in the original 2D representation. So ∞ .

3-5. عنوان بخش

Check separability:

- $[x,y] \rightarrow [y,x]$: just swaps axes $\Rightarrow$ still not separable.
- $[x,y] \rightarrow [x,y,1]$: adds bias but still cannot separate the ± 1 vs ± 2 on both sides consistently $\Rightarrow$ not separable.
- $[x,y] \rightarrow [x,y,x^2,1]$: does separate (because x^2 distinguishes $|x|=1$ from $|x|=2$, and the constant enables shifting).
- $[x,y] \rightarrow [x,y,x^2 + y^2]$: no constant; constraints force contradiction $\Rightarrow$ not separable.

4-5. عنوان بخش

A simple separator is to use only the radius-squared feature:

$$w=[0,0,1,-2]$$

Then score is $(x^2 + y^2) - 2$:

- Negative points have $x^2 + y^2 = 1 \Rightarrow 1 - 2 = -1 < 0$
- Positive points have $x^2 + y^2 = 4 \Rightarrow 4 - 2 = 2 > 0$

5-5. عنوان بخش

- $[x,y] \rightarrow [x,y,x^4,1]$

پرسش 6 - عنوان پرسش

Which functions each computation graph can represent

- G1 (multiply only): linear through origin $\Rightarrow$ (a)
- G2 (affine): $w^T x + b \Rightarrow$ (a), (b)
- G3 (ReLU of affine): $\max(0, wx + b) \Rightarrow$ (c), (d)
- G4 (two affines in a row): still affine $\Rightarrow$ (a), (b)
- G5 (affine $\rightarrow$ ReLU $\rightarrow$ affine): one-kink piecewise linear with a constant plateau on one side $\Rightarrow$ (c), (d), (e)

- G6 (two ReLUs in sequence): can create two breakpoints in a clipped way $\Rightarrow$ (f) (and can also do (c),(d),(e))
- G7 (two ReLUs in parallel + sum): general 2-kink piecewise linear $\Rightarrow$ (a), (b), (c), (d), (e), (f), (h)
Not (g) exactly (since log is not piecewise linear).

G1: output =wx

- (a) 2x: set w=2

G2: output =wx+b

- (a) 2x: w=2, b=0
- (b) 4x-5: w=4, b=-5

G3: output =ReLU(wx+b)

- (c) max(0,2x-5): w=2, b=-5
- (d) max(0,-2x-5): w=-2, b=-5

G4: output = $w_2(w_1x + b_1) + b_2$ (equivalent to affine)

One simple parameterization:

- (a) 2x: $w_1 = 1, b_1 = 0, w_2 = 2, b_2 = 0$
- (b) 4x-5: $w_1 = 1, b_1 = 0, w_2 = 4, b_2 = -5$

G5: output = $w_2 \text{ReLU}(w_1x + b_1) + b_2$

- (c) max (0, 2x - 5): $w_1 = 2, b_1 = -5, w_2 = 1, b_2 = 0$
- (d) max(0,-2x-5): $w_1 = -2, b_1 = -5, w_2 = 1, b_2 = 0$
- (e) 1 for $x < 2$, -x+3 for $x \geq 2$:
Use 1-ReLU(x-2):
 $w_1 = 1, b_1 = -2, w_2 = -1, b_2 = 1$

7-1. عنوان بخش

For one-of-K targets $t_{kn} \in \{0, 1\}$ with $\sum_k t_{kn} = 1$, model outputs $y_k(x_n, w) = p(t_k = 1 | x_n)$

Likelihood:

$$p(\{t_n\} | \{x_n\}, w) = \prod_{n=1}^N \prod_{k=1}^K y_k(x_n, w)^{t_{kn}}$$

Log-likelihood:

$$\ln p = \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w)$$

Maximizing log-likelihood is equivalent to minimizing negative log-likelihood:

$$E(w) = - \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w)$$

which is exactly the multiclass cross-entropy.

7-2. عنوان بخش

Let model output $y = p(t=1 | x)$. Observed label is flipped with probability ϵ .

Then:

$$\begin{aligned} p(\tilde{t} = 1 | x) &= (1 - \epsilon)y + \epsilon(1 - y) = \epsilon + (1 - 2\epsilon)y \\ p(\tilde{t} = 0 | x) &= (1 - \epsilon)(1 - y) + \epsilon y = (1 - \epsilon) - (1 - 2\epsilon)y \end{aligned}$$

Negative log-likelihood over data $\{(x_n, \tilde{t}_n)\}$:

$$E(w) = - \sum_{n=1}^N [\tilde{t}_n \ln(\epsilon + (1 - 2\epsilon)y_n) + (1 - \tilde{t}_n) \ln((1 - \epsilon) - (1 - 2\epsilon)y_n)]$$

Where $y_n = y(x_n, w)$

If $\epsilon=0$, this becomes:

$$E(w) = - \sum_{n=1}^N [\tilde{t}_n \ln y_n + (1 - \tilde{t}_n) \ln(1 - y_n)]$$

the usual binary cross-entropy.

7-3. عنوان بخش

A natural probabilistic mapping is:

$$p(t = +1|x) = \frac{1+y}{2} p(t = -1|x) = \frac{1-y}{2}$$

then the negative log-likelihood (cross-entropy in ± 1 coding) is:

$$E(w) = - \sum_{n=1}^N \left[\frac{1+t_n}{2} \ln\left(\frac{1+t_n}{2}\right) + \frac{1-t_n}{2} \ln\left(\frac{1-t_n}{2}\right) \right]$$

Where $y_n = y(x_n, w) \in [-1, +1]$

Appropriate activation: choose an output activation that maps $\mathbb{R} \rightarrow (-1, 1)$ e.g.

$$y = \tanh(a) \text{ or } \tanh\left(\frac{a}{2}\right)$$

پرسش 8 - عنوان پرسش

(1) Distribution of z and σ_z^2

Mean:

$$E[w^\top x] = 0 \Rightarrow E[z] = b$$

Variance:

$$\text{Var}(w^\top x) = \sum_{i=1}^n \text{Var}(w_i x_i)$$

with independence and $E[w_i] = E[x_i] = 0$:

$$\text{Var}(w_i x_i) = E[w_i^2] E[x_i^2] = \sigma_w^2 \sigma_x^2$$

$$\sigma_z^2 = \text{Var}(z) = \text{Var}(w^\top x) = n \sigma_w^2 \sigma_x^2.$$

Thus:

$$z \sim N(b, \sigma_z^2), \sigma_z^2 = n \sigma_w^2 \sigma_x^2.$$

(2) Activation probability

$$P(a > 0) = P(z > 0) = P\left(\frac{z-b}{\sigma_z} > -\frac{b}{\sigma_z}\right) = 1 - \Phi\left(-\frac{b}{\sigma_z}\right) = \Phi\left(\frac{b}{\sigma_z}\right)$$

(3) Bias $b = -3\sigma_z$:

$$P(a > 0) = \Phi(-3) \approx 0.00135.$$

With $N=1000$ neurons, expected active:

$$1000 \times 0.00135 \approx 1.35 P(a > 0) \approx 0.00135 (\approx 1 - 2 \text{neurons})$$

(4) Zero-gradient proof and implication

If for all samples in a mini-batch $z \leq 0$, then $\frac{\partial a}{\partial z} = 0$ (given subgradient rule).

For any loss L ,

$$\frac{\partial L}{\partial w} = \sum_{\text{batch}} \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w} = \sum_{\text{batch}} \frac{\partial L}{\partial a} \cdot 0 \cdot x = 0.$$

$$\text{So } \Delta w = -\eta \frac{\partial L}{\partial w} = 0.$$

Implication: with this ReLU derivative choice, a neuron that is inactive ($z \leq 0$) on every batch sample receives no gradient signal and cannot be resurrected by standard gradient descent unless some change elsewhere causes z to become positive on some data (or you use leaky ReLU / different subgradient / noise mechanisms).

پرسش 9 - عنوان پرسش

(1) Symmetric constraints

Forward: For one output unit

$$y_j = \sum_{i=1}^{n_{in}} W_{ji} x_i$$

Assuming independent, zero-mean, identical variance $\text{Var}(x_i) = v$,

$$\text{Var}(y_j) = \sum_{i=1}^{n_{in}} \text{Var}(W_{ji} x_i) = n_{in} \text{Var}(W) v.$$

To keep $\text{Var}(y_j) = v$, require:

$$n_{in} \text{Var}(W) = 1 \Rightarrow \text{Var}(W) = \frac{1}{n_{in}}.$$

Backward: A typical backprop step has gradients summed through weights; similarly,

$$\text{Var}(\delta) = n_{out} \text{Var}(W) \text{Var}(\delta_{next}).$$

To preserve gradient variance:

$$n_{out} \text{Var}(W) = 1 \Rightarrow \text{Var}(W) = \frac{1}{n_{out}}.$$

(2) Xavier compromise

Compromise between $\frac{1}{n_{in}}$ and $\frac{1}{n_{out}}$ by matching the average:

$$\sigma^2 = \text{Var}(W) = \frac{2}{n_{out} + n_{in}}.$$

(3) Depth-induced variance (gain)

Given layer gain $G = n \cdot \text{Var}(W)$ (for constant width n):

$$\text{Var}(y^{(l)}) = G \text{Var}(x^{(l-1)}).$$

By recursion over L layers:

$$\text{Var}(y^{(L)}) = G^L \text{Var}(x^{(0)}).$$

(4) Stability bound with error ϵ

If $G = 1 + \epsilon$, then

$$\text{Var}(y^{(L)}) = (1 + \epsilon)^L \text{Var}(x^{(0)})$$

which grows (or decays) exponentially with depth L .

(5) Numerical amplification for $L=150$, $\epsilon=0.02$

$$(1.02)^{150} \approx 19.5. \text{ (amplification factor)}$$

Why risky in FP16: FP16 has limited dynamic range and coarse mantissa precision; exponential growth in activations/gradients can cause overflow (∞), underflow to 0, and large rounding errors—destabilizing training.

پرسش 10 - عنوان پرسش

First I will present the outputs and then analyze them.

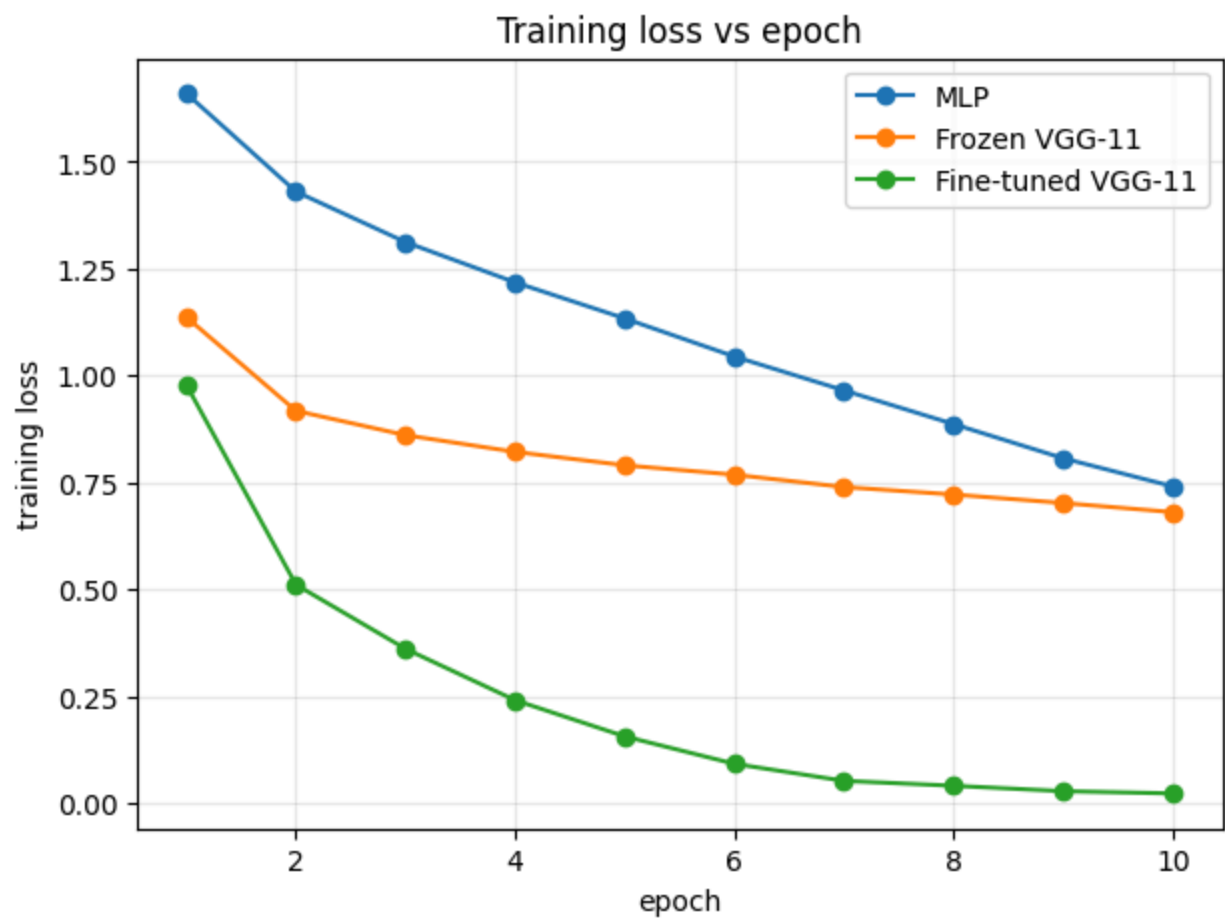


Fig 4. Training loss VS. epoch

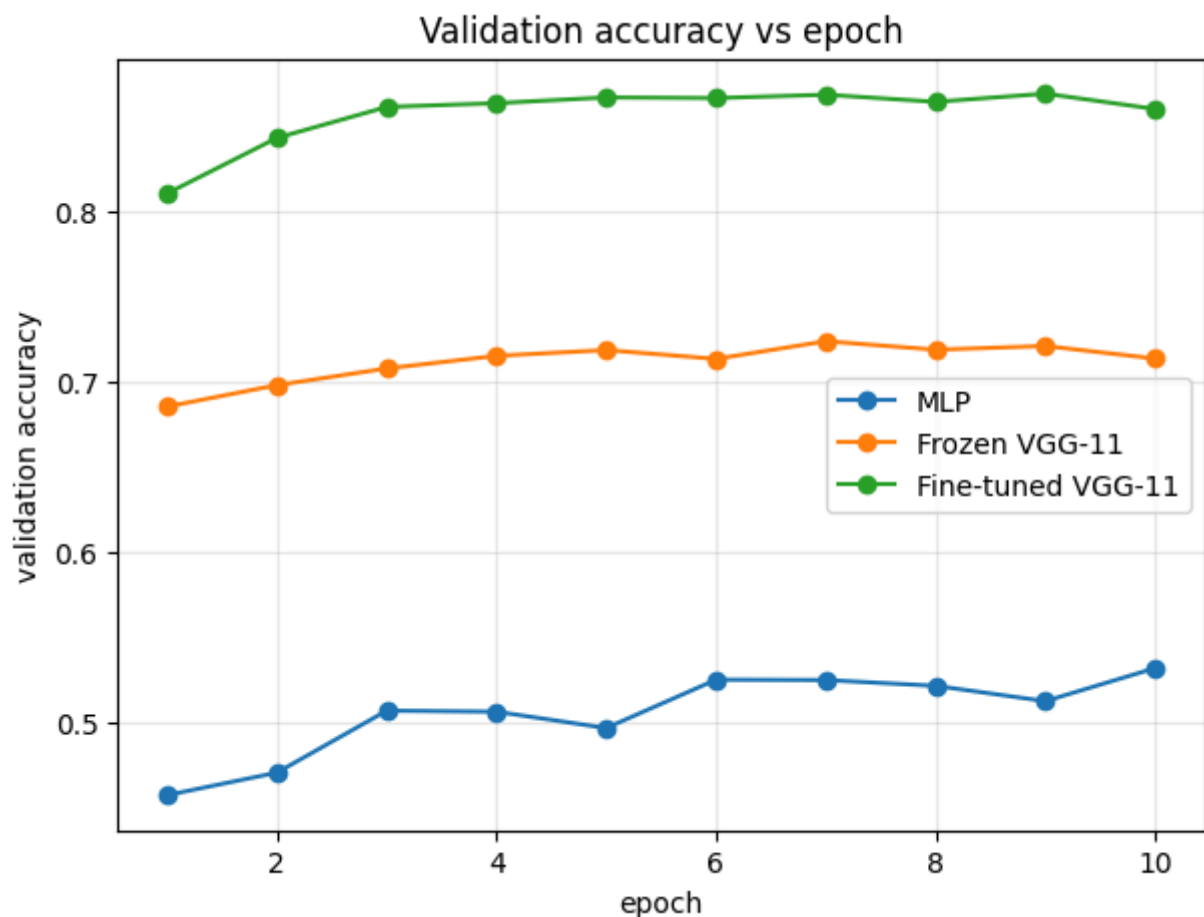


Fig 5. Validation accuracy VS. epoch

The MLP performs worst because flattening pixels destroys spatial structure; it has no convolutional inductive bias (locality/translation invariance), so it learns weak features and generalizes poorly (~53%).

Why pretrained init is faster + more stable:

Pretrained VGG weights already encode strong, general-purpose image features (edges, corners, textures, simple shapes). So optimization begins in a *good region* of parameter space: gradients point toward refining useful features instead of “discovering vision from scratch.” This reduces the amount of parameter searching needed, gives better-conditioned learning signals, and avoids the noisy early-stage instability that happens when a randomly initialized network is trying to invent feature detectors. That’s why VGG drops faster and has smoother, more reliable improvement than the MLP trained from scratch.

Frozen VGG-11 performs much better (~72%) because the pretrained convolution layers already extract strong generic visual features (edges/textures/shapes). Training only the classifier head is enough to adapt those features to CIFAR-10.

Generalization Gap (Frozen VGG beats MLP)

Even with frozen convolution weights, Frozen VGG reaches about 0.71–0.72 validation accuracy, while the MLP reaches only about 0.53.

Why this happens (spatial inductive bias):

- A CNN assumes and exploits image structure:
 - **Local connectivity:** early filters look at local neighborhoods (small patches), which is exactly where edges and textures live.
 - **Weight sharing:** the same filter is applied across the whole image, so the model efficiently detects the same pattern anywhere (translation handling).
 - **Hierarchical features:** stacking conv layers builds from edges → textures → parts → objects.
- A **flattened MLP** has **no built-in notion of locality or translation**. Flattening destroys the 2D structure, so the MLP must learn “from scratch” that nearby pixels are related and that the same pattern can appear in different positions. With limited data/epochs, it tends to overfit inefficiently and generalize poorly.

So the Frozen VGG performs far better because its pretrained CNN feature extractor already produces a strong, spatially meaningful representation; the classifier head only needs to map those features to CIFAR-10 classes.

Fine-tuning VGG-11 performs best (~86%) because updating all layers lets the pretrained filters specialize to CIFAR-10, improving both optimization (fast loss drop) and generalization. The slight late-epoch dip suggests mild overfitting, but it’s small compared to the overall gain.

1) Training loss behavior (convergence speed)

- **MLP (from scratch):** training loss decreases steadily from **1.66** → **0.74**, but stays relatively high after 10 epochs. This indicates slower learning and limited capacity/inductive bias for image structure.
- **Frozen VGG-11 (train head only):** loss drops from **1.14** → **0.68**, improving faster than the MLP even though most weights are fixed. This shows the pretrained convolution features are already useful; only a lightweight classifier needs to adapt.
- **Fine-tuned VGG-11:** loss collapses quickly **0.98** → **0.0235**, the fastest and lowest of all three. Fine-tuning lets the pretrained filters adapt to CIFAR-10, so the model fits the

training data extremely well.

Overall: Fine-tuning converges fastest and reaches the lowest training loss, frozen VGG is next, MLP is slowest.

2) Validation accuracy behavior (generalization)

Validation accuracy trends:

- **MLP:** rises from **0.457** → **0.532**, with small ups/downs (e.g., dip around epoch 5). Final val acc ≈ **53%**.
- **Frozen VGG-11:** starts much higher (**0.685**) and stabilizes around **0.71–0.724**, peaking at **0.7238** (epoch 7). Final val acc ≈ **71%**.
- **Fine-tuned VGG-11:** starts high (**0.811**), climbs to about **0.869** (peak epoch 9), and ends at **0.860**. Final val acc ≈ **86%**.

Best model by validation accuracy: Fine-tuned VGG-11 (~86–87%).

3) Evidence of underfitting vs overfitting

- **MLP** looks like **underfitting / limited representation**: even after 10 epochs, both training loss and validation accuracy suggest it can't learn strong image features from flattened pixels.
- **Frozen VGG-11** shows **good generalization, limited capacity**: training loss decreases but can't go very low because features are fixed. Validation plateaus around ~72%.
- **Fine-tuned VGG-11** shows **very strong fit + strong generalization**: training loss becomes extremely low, while validation remains high. There is a **small sign of mild overfitting** late (val peaks ~0.869 then ends ~0.860), but the drop is small, so generalization is still excellent.

4) Test accuracy (final comparison)

Our test accuracies:

- **MLP: 0.5333**
- **Frozen VGG-11: 0.7158**
- **Fine-tuned VGG-11: 0.8582**

This matches the validation trends and confirms the ranking:

Fine-tuned VGG > Frozen VGG > MLP.

جدول 1. عنوان جدول نمونه

ML Model	Role In Our Life
Naïve Bayes	Makes us overconfident
Decision Tree	Makes us comfortable
SVM	Makes us question our life choices (e.g. taking this (course

Model Performance
on Training Data



Model Performance
on Test Data



شكل 1. عنوان تصوير نمونه

در این بخش، منابعی که در تمرینات و پیاده‌سازی استفاده کرده‌اید (مانند مقالات، سایت‌ها و ...) را در یک فرمت استاندارد فهرست کنید. برای رفرنس‌دهی می‌توانید از بخش Reference در نرم‌افزار Word، سایت‌های مخصوص یا افزونه‌های مرتبط استفاده کنید. همچنین، لازم است که هر منبع استفاده‌شده را در بخش مربوطه ارجاع دهید. توجه داشته باشید که شباهت کد یا پاسخ شما به منابع دیگر باید در حد معقول باشد؛ کپی‌برداری مستقیم از منابع، مدنظر ما نیست. در صورت مشاهده کپی کامل از یک منبع، نمره آن تمرین یا بخش مربوطه به شما تعلق نخواهد گرفت.